

Scala/Java

Exercise 1 - stateless into stateful conversion

I'm starting the implementation with setting UTC as my timezone. I often get confused while I debug the jobs because of the display difference that is in my local time zone and the real data that is in UTC.

```
val sparkSession = SparkSession.builder().master("local[*]")
  .config("spark.sql.shuffle.partitions", 2)
  .config("spark.sql.session.timeZone", "UTC")
  .getOrCreate()
TimeZone.setDefault(TimeZone.getTimeZone("UTC"))
import sparkSession.implicits._
```

The reading part is just after:

```
val inputStream = sparkSession.readStream.format("kafka")
  .options(Map(
    "kafka.bootstrap.servers" -> "localhost:29092",
    "subscribe" -> InputTopic,
    "startingOffsets" -> "EARLIEST"
  )).load()

val inputVisits = inputStream.selectExpr("CAST(value AS STRING) AS value",
  "timestamp")
  .select(functions.from_json($"value".cast("string"),
    UserVisitWithTimestamp.SchemaAsStructType).as("visit"),
    $"timestamp")
  .selectExpr("visit.*", "timestamp")
  .as[UserVisitWithTimestamp]
```

There is no rocket science here but 2 important things to notice:

1. The timestamp attribute. It represents Apache Kafka append time, so when the data arrived to the topic. It's immutable and will be used to generate the session id.
2. The UserVisitWithTimestamp case class. I'm casting the dataset explicitly to facilitate the processing part in the stateful function. Otherwise, I should have been working on the low-level Row object which is highly impractical.

Next comes the stateful definition part:

```
val validVisits = inputVisits.filter(visit => {
  visit.visitedPage != null && visit.eventTime != null
})
val sessionTimeout = TimeUnit.MINUTES.toMillis(16)
val userVisits: KeyValueGroupedDataset[Int, UserVisitWithTimestamp] = validVisits
  .withWatermark("eventTime", "15 minutes")
```

```

    .groupByKey(_._userId)
    val userSessions: Dataset[Option[VisitOutput]] = userVisits.mapGroupsWithState(
        GroupStateTimeout.EventTimeTimeout()
    )(
        VisitsMapper.generateVisitDuration(sessionTimeout)
    )

```

It's quite self explanatory. I'm filtering irrelevant events, later defining the watermark, grouping records by `userId` and finally, calling the stateful mapping function. This stateful mapping function is a mix of incremental and cumulative logic. First, I'm processing data for already existing session:

```

val stateToUpdate = if (currentState.exists) {
    currentState.get.copy(
        pages = currentState.get.pages ++ visitEvents.map(visit =>
            Visit.fromUserVisitWithTimestamp(visit))
    )
}

```

In that case, I only need to add new pages since the previous attributes were resolved during the state initialization. The initialization that looks like that:

```

var browserCode = ""
var browserVersion = ""
var firstEventTime = Long.MaxValue
val newPages = visitEvents.map(visit => {
    browserCode = visit.browserKey
    browserVersion = visit.browserVersion
    firstEventTime = Math.min(firstEventTime, visit.timestamp.getTime)
    Visit(visit.visitedPage, visit.eventTime)
}).toSeq
VisitState(
    firstEventTimeEpochMillis = firstEventTime, pages = newPages,
    browserCode = browserCode, browserVersion = browserVersion
)

```

The snippet represents the stateful logic mix. First, the `browserCode`, `browserVersion` and `firstEventTime` are the examples of the incremental state. I keep only one copy of them. Next, the `Visits` are the cumulative state because I need them to build correct navigation history.

In the end, I'm resolving the watermark base value because the first micro-batch doesn't have the watermark set. It always happens at the end of the micro-batch. As a result, if I used the watermark value here, my state would expire just in the next micro-batch:

```

val baseWatermark: Long = if (currentState.getCurrentWatermarkMs() > 0L) {
    currentState.getCurrentWatermarkMs()
} else {
    stateToUpdate.pages.maxBy(visit => visit.time.getTime).time.getTime
}
currentState.setTimeoutTimestamp(baseWatermark + timeoutDurationMs)

```

```
None
```

I'm finally returning nothing so that any pending session doesn't get printed to the output sink. The stateful function also has a part to generate the output, though. But it's only called on the expired state:

```
if (currentState.hasTimedOut) {  
  val visitDuration = currentState.get.toVisitOutput(userId)  
  currentState.remove()  
  Some(visitDuration)  
}
```

If we return to the job, the next part is filtering all pending sessions and joining the outcome with the reference master dataset:

```
val technicalReferenceDataset = sparkSession.read  
  .schema("id STRING, name STRING, version STRING")  
  .json(ReferenceDatasetFileName)  
  
val sessionsToWrite = userSessions.filter(session => session.isDefined)  
  .map(session => session.get)  
  .join(technicalReferenceDataset,  
    functions.expr("""  
      |id = browserCode AND version = browserVersion  
      |""".stripMargin), "left_outer")  
  .withColumn("technicalContext", functions.struct($"name", $"version"))  
  .drop("browserCode", "browserVersion", "id", "name", "version")  
  .selectExpr("TO_JSON(STRUCT(*)) AS value")
```

In the end, the so formatted value is written to the Apache Kafka sink:

```
val writeQuery = sessionsToWrite.writeStream  
  .outputMode(OutputMode.Update())  
  .format("kafka")  
  .options(  
    Map(  
      "checkpointLocation" -> "/tmp/bde/6/e1/checkpoint/stateful",  
      "kafka.bootstrap.servers" -> "localhost:29092",  
      "topic" -> OutputTopic  
    )  
  )  
  .start()  
  
writeQuery.awaitTermination()
```

Exercise 2 - partial results trigger

All the triggers implement this class:

```
public abstract class Trigger<T, W extends Window> implements Serializable
```

The class provides various callbacks that are invoked depending on the situation:

- `onElement` gets called when an event enters into the pane
- `onProcessingTime` gets called for the processing time triggers
- `onEventTime` gets called for the event time (watermark) triggers

For our homework the most relevant callback is the first one. Why not the others? We don't rely on the processing time, it would be too risky. Also, we can't rely on the event time because it means no more no less that the results get emitted when the watermark moves on. Instead, the partial results emission logic must be implemented in the `onElement`:

```
private static final long PARTIAL_RESULTS_MILLIS_THRESHOLD =
    TimeUnit.MINUTES.toMillis(3L);
private final EventTimeTrigger eventTimeTrigger = EventTimeTrigger.create();

@Override
public TriggerResult onElement(Object element, long watermarkTime, TimeWindow
    window, TriggerContext triggerContext) throws Exception {
    // let's check now what are the partial results we should emit
    long remainingMillisBeforeWindowEnd = window.maxTimestamp() - watermarkTime;
    if (remainingMillisBeforeWindowEnd <= PARTIAL_RESULTS_MILLIS_THRESHOLD) {
        LOG.info("Generating partial results because the difference is
            "+remainingMillisBeforeWindowEnd+ " ms");
        return TriggerResult.FIRE;
    }
    return eventTimeTrigger.onElement(element, watermarkTime, window,
        triggerContext);
}
```

Do you notice something? Yes, the event time trigger. I just wrote we don't want to use it but it turns out, we don't want to use it only for the partial results. Otherwise, it's the best candidate to cover other scenarios. That's why it's referenced pretty much everywhere as the default implementation:

```
@Override
public TriggerResult onProcessingTime(long watermarkTime, TimeWindow window,
    TriggerContext triggerContext) throws Exception {
    return eventTimeTrigger.onProcessingTime(watermarkTime, window, triggerContext);
}

@Override
public TriggerResult onEventTime(long watermarkTime, TimeWindow window,
    TriggerContext triggerContext) {
    LOG.info("Triggering on the event time");
    return eventTimeTrigger.onEventTime(watermarkTime, window, triggerContext);
}

@Override
public void clear(TimeWindow window, TriggerContext triggerContext) throws Exception
{
}
```

```
eventTimeTrigger.clear(window, triggerContext);
}
```

Why am I not extending the EventTimeTrigger directly in my trigger? The constructor of this class is private and the compilation returns an error our the constructor unavailability:

```
@PublicEvolving
public class EventTimeTrigger extends Trigger<Object, TimeWindow> {
    private static final long serialVersionUID = 1L;

    private EventTimeTrigger() {}
}
```

Exercise 3 - window-based counters

This announcement starts with the data source declaration, again with the UTC setting for easier debugging:

```
val sparkSession = SparkSession.builder().master("local[*]")
    .config("spark.sql.session.timeZone", "UTC")
    .getOrCreate()
TimeZone.setDefault(TimeZone.getTimeZone("UTC"))
import sparkSession.implicits._

val inputStream = sparkSession.readStream.format("kafka")
    .options(Map(
        "kafka.bootstrap.servers" -> "localhost:29092",
        "subscribe" -> InputTopic,
        "startingOffsets" -> "EARLIEST",
    )) .load()
```

Later, the job implements the window logic:

```
val windowsWithBrowserCount = inputStream
    .select(functions.from_json(
        $"value".cast("string"), "eventTime TIMESTAMP, browser STRING", Map.empty[String, String])
        .as("value"))
    .selectExpr("value.*")
    .withWatermark("eventTime", "5 minutes")
    .groupBy($"browser", functions.window($"eventTime", "5 minutes"))
    .agg("eventTime" -> "max", "browser" -> "count")
    .selectExpr("browser", "window",
        "`count(browser)` AS count",
        "CAST(window.end - `max(eventTime)` AS LONG) AS completeness")
```

The logic relies on Apache Spark API and hence is quite self-explanatory. The single tricky point here is the reference of the aggregation in the select expression. I had to wrap them with backticks. Otherwise the query planner would consider this as an aggregation `max(eventTime)` which obviously would fail since there is no grouping involved before.

In the end, I'm writing all the results back to Apache Kafka:

```
val writeQuery = windowsWithBrowserCount
  .selectExpr("TO_JSON(STRUCT(*)) AS value")
  .writeStream.outputMode("update")
  .format("kafka")
  .options(Map(
    "checkpointLocation" -> "/tmp/bde/6/e3/checkpoint",
    "kafka.bootstrap.servers" -> "localhost:29092",
    "topic"-> OutputTopic
  )).start()

writeQuery.awaitTermination()
```

Python

Exercise 1 - stateless into stateful conversion

I'm starting the implementation with data source reading:

```
input_data = spark_session.readStream.format("kafka") \
  .option("kafka.bootstrap.servers", "localhost:29092") \
  .option("subscribe", get_input_topic_name()) \
  .option("startingOffsets", "EARLIEST") \
  .load()

input_visits = input_data.selectExpr('CAST(value AS STRING)', 'timestamp') \
  .select(F.from_json(F.col('value'), "userId INT, eventTime TIMESTAMP, browserKey STRING, browserVersion STRING, visitedPage STRING")
    .alias('data'), 'timestamp') \
  .selectExpr('data.*', 'timestamp')
```

There is no rocket science here but one important thing to notice. The timestamp attribute. It represents Apache Kafka append time, so when the data arrived to the topic. It's immutable and will be used to generate the session id.

Next comes the stateful definition part:

```
valid_visits = input_visits.filter('visitedPage IS NOT NULL AND eventTime IS NOT NULL')
user_visits = valid_visits.withWatermark('eventTime', '15 minutes') \
  .groupBy(F.col("userId"))

user_sessions = user_visits.applyInPandasWithState(
  func=generate_visit_output,
  outputStructType=StructType([
    StructField("sessionId", StringType()),
    StructField("userId", IntegerType()),
  ])
```

```

        StructField("startTime", TimestampType()),
        StructField("endTime", TimestampType()),
        StructField("browserCode", StringType()),
        StructField("browserVersion", StringType()),
        StructField("navigation", ArrayType(StructType([
            StructField("page", StringType()), StructField("timeSpent", LongType())
        ])))
    ),
    stateStructType=StructType([
        StructField("firstEventTimeEpochMillis", LongType()),
        StructField("pages", ArrayType(StructType([
            StructField("visitedPage", StringType()),
            StructField("eventTimeAsMilliseconds", LongType())
        ]))),
        StructField("browserCode", StringType()),
        StructField("browserVersion", StringType())
    ]),
    outputMode="update",
    timeoutConf="EventTimeTimeout"
)

```

It's quite self explanatory. I'm filtering irrelevant events, later defining the watermark, grouping records by userId and finally, calling the stateful mapping function. This stateful mapping function is a mix of incremental and cumulative logic. First, I'm processing new data:

```

new_pages = []
first_event_time_as_milliseconds = 0
browser_code = ''
browser_version = ''
for input_df_for_group in visit_events:
    browser_code = input_df_for_group['browserKey'].iloc[0]
    browser_version = input_df_for_group['browserVersion'].iloc[0]
    input_df_for_group['eventTimeAsMilliseconds'] = input_df_for_group['eventTime'] \
        .apply(lambda x: int(pandas.Timestamp(x).timestamp()) * 1000)
    first_event_time_as_milliseconds = int(
        input_df_for_group['eventTimeAsMilliseconds'].min()
    )
    if should_use_event_time_for_watermark:
        base_watermark = int(input_df_for_group['eventTimeAsMilliseconds'].max())

    new_pages = input_df_for_group[['visitedPage',
        'eventTimeAsMilliseconds']].to_dict(orient='records')

```

The snippet represents the logic mix. First, the browser_code, browser_version and first_event_time_as_milliseconds are the examples of the incremental state. I keep only one copy of them. Next, the new_pages are the cumulative state because I need them to build correct navigation history. Depending on the existence of the state, I'm either updating all of these attributes or only the pages:

```

old_pages_from_state = []
if current_state.exists:
    first_event_time_as_milliseconds, old_pages_from_state, browser_code,
    browser_version = current_state.get

```

```
pages_for_state = new_pages + old_pages_from_state
current_state.update((first_event_time_as_milliseconds, pages_for_state,
                      browser_code, browser_version,))
```

In the end, I'm resolving the watermark base value because the first micro-batch doesn't have the watermark set. It always happens at the end of the micro-batch. As a result, if I used the watermark value here, my state would expire just in the next micro-batch:

```
should_use_event_time_for_watermark = current_state.getCurrentWatermarkMs() == 0
base_watermark = current_state.getCurrentWatermarkMs()
# ...
timeout_timestamp = base_watermark + session_expiration_time_16min_as_ms
current_state.setTimeoutTimestamp(timeout_timestamp)
```

I'm finally returning nothing or something, depending on the state expiration:

```
visit_to_return = None
if current_state.hasTimedOut:
    visit_to_return = get_visit_duration_to_return(current_state.get())
    current_state.remove()

# ...
if visit_to_return:
    yield pandas.DataFrame(visit_to_return)
```

If we return to the job, the next part is joining the defined sessions with the reference master dataset:

```
technical_reference_dataset = spark_session.read.schema('id STRING, name STRING,
version STRING') \
    .json(get_reference_dataset_location())

session_to_write = user_sessions.join(technical_reference_dataset,
                                     F.expr('id = browserCode AND version =
browserVersion'), 'left_outer') \
    .withColumn('technicalContext', F.struct('name', 'version')) \
    .drop('browserCode', 'browserVersion', 'id', 'name', 'version') \
    .selectExpr('TO_JSON(STRUCT(*)) AS value')
```

In the end, the so formatted value is written to the Apache Kafka sink:

```
write_query = session_to_write.writeStream.outputMode("update") \
    .option('checkpointLocation', "/tmp/bde/6/e1/checkpoint/stateful/") \
    .format('kafka') \
    .option('kafka.bootstrap.servers', 'localhost:29092') \
    .option('topic', get_output_topic_name()) \
    .start()
```



```
write_query.awaitTermination()
```

Exercise 2 - partial results trigger

All the triggers implement this class:

```
class Trigger(ABC, Generic[T, W]):
```

The class provides various callbacks that are invoked depending on the situation:

- `on_element` gets called when an event enters into the pane
- `on_processing_time` gets called for the processing time triggers
- `on_event_time` gets called for the event time (watermark) triggers

For our homework the most relevant callback is the first one. Why not the others? We don't rely on the processing time, it would be too risky. Also, we can't rely on the event time because it means no more no less that the results get emitted when the watermark moves on. Instead, the partial results emission logic must be implemented in the `on_element`:

This time, I'm directly inheriting the `EventTimeTrigger` class:

```
class PartialEventTimeWindowTrigger(EventTimeTrigger):
```

And that's why I can only focus on implementing the `on_element` function that fall backs into the `EventTimeTrigger`:

```
PARTIAL_RESULTS_MILLIS_THRESHOLD_3_MIN_AS_MILLIS = 3 * 60 * 1000

def on_element(self, element: T, watermark_time: int, window: W, ctx:
    'Trigger.TriggerContext') -> TriggerResult:
    remaining_millis_before_window_end = (window.max_timestamp() - watermark_time)
    if remaining_millis_before_window_end <=
PartialEventTimeWindowTrigger.PARTIAL_RESULTS_MILLIS_THRESHOLD_3_MIN_AS_MILLIS:
        print("Emitting partial results")
        return TriggerResult.FIRE
    else:
        return super().on_element(element, watermark_time, window, ctx)
```

Exercise 3 - window-based counters

This announcement starts with the data source declaration:

```
input_data = spark_session.readStream.format("kafka") \
    .option("kafka.bootstrap.servers", "localhost:29092") \
    .option("subscribe", get_input_topic_name()) \
    .option("startingOffsets", "EARLIEST") \
    .load()
```

Later, the job implements the window logic:

```
windows_with_browser_groups = input_data \
    .select(F.from_json(F.col('value').cast('string'), 'eventTime TIMESTAMP, browser STRING').alias('value')) \
    .selectExpr('value.*') \
    .withWatermark('eventTime', '5 minutes') \
    .groupBy('browser', F.window(F.col("eventTime"), "5 minutes"))

windows_with_browser_aggregation = windows_with_browser_groups.agg(
    F.max('eventTime'), F.count('browser')
)

output_dataframe = windows_with_browser_aggregation.selectExpr('browser', 'window',
    'count(browser)` AS count',
    'CAST(window.end - `max(eventTime)` AS LONG) AS completeness')
```

The logic relies on Apache Spark API and hence is quite self-explanatory. The single tricky point here is the reference of the aggregation in the select expression. I had to wrap them with backticks. Otherwise the query planner would consider this as an aggregation `max(eventTime)` which obviously would fail since there is no grouping involved before.

In the end, I'm writing all the results back to Apache Kafka:

```
write_query = output_dataframe.selectExpr('TO_JSON(STRUCT(*)) AS value') \
    .writeStream.outputMode('update') \
    .format('kafka') \
    .option('checkpointLocation', '/tmp/bde/6/e3/checkpoint') \
    .option('kafka.bootstrap.servers', 'localhost:29092') \
    .option('topic', get_output_topic_name())

write_query.start().awaitTermination()
```

Bartosz.